

Maze Solver Robot

Project Proposal — Autonomous Maze Exploration and Shortest-Path Solving on ATmega32

Md. Ariful Islam Hasan Ferdous Mahee B. M. Sufian Ashraf

Department of Computer Science and Engineering

August 9, 2026

Outline

1. Motivation
2. Equipment
3. System Design
4. Role of the ATmega32
5. Maze Mapping and Algorithm
6. Implementation Plan
7. Use of Other Controllers
8. Testing and Success Criteria
9. Challenges and Future Scope

1. Motivation

Problem Statement

What is the Problem?

Autonomous navigation through an **unknown environment** is a core problem in mobile robotics — search-and-rescue robots, warehouse pickers, and cleaning robots all need to explore, build an internal map, and find an efficient path without any prior knowledge of the layout or human guidance.

Why is it Important?

- Core capability behind search-and-rescue robotics
- Foundation of SLAM and exploration algorithms
- Directly transfers to warehouse/indoor navigation

Why This Project?

- Combines real-time control with classical search algorithms
- Forces on-the-fly mapping under memory constraints
- Natural, well-scoped extension of embedded control (ATmega32)

Project Objectives

Main Objectives

1. **Design and build** an autonomous robot capable of exploring an unknown maze of unit cells using wall-following sensors.
2. **Implement wall detection** using ultrasonic sensors on the front, left, and right of the robot.
3. **Implement an exploration algorithm** (flood-fill) that incrementally maps the maze and updates the shortest known path to the goal.
4. **Implement a run-time architecture:** explore the maze on the first run, then execute the fastest known path on a speed run.
5. **Develop precise motor control** with PWM speed regulation and closed-loop odometry via optical wheel encoders for accurate cell-to-cell moves and turns.
6. **Handle the dead-end edge case:** detect dead ends during exploration and backtrack correctly to continue the search.

2. Equipment

Hardware Components

Component	Purpose
ATmega32A	Central microcontroller; runs flood-fill, wall sensing, and motor control
3x HC-SR04 Ultrasonic Sensors	Front, left, and right wall detection at each cell
2x Wheel Encoders (Optical)	Optical encoder discs + sensors on each wheel for precise distance and turn odometry
L298N Motor Driver Module	Dual H-bridge module for PWM motor speed and direction control
DC TT Gear Motors	Provides differential drive locomotion
2WD Chassis + Wheels	Mechanical frame, wheels, and caster for mobility and stability
Status LEDs	Visual indication of robot state (exploring, dead-end, goal reached, speed run)
L7805 Voltage Regulator	Stabilizes 5V supply for MCU, sensors, and logic
3.7V 18650 Li-Ion Batteries (x3)	3S configuration providing 11.1V main power supply
18650 Battery Holder (3S)	Houses batteries and supplies power to driver and regulator
Crystal Oscillator (16 MHz)	External clock source for ATmega32A system clock
AVR ISP Programmer	In-System Programmer (ISP) for flashing code onto ATmega32A

Development Tools

Software and Programming Tools

- **Atmel Studio 7.0 / AVR-GCC** — Writing and compiling the embedded C program
- **AVRDUDE** — Uploading the compiled firmware to the ATmega32
- **Git** — Version control and collaboration for the source code

Hardware Development Tools

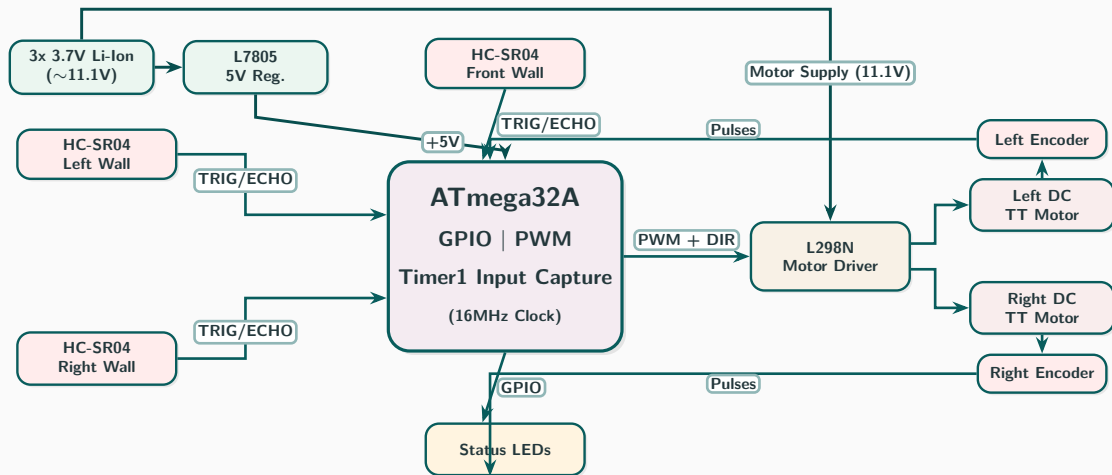
- **AVR ISP Programmer** — Programming the ATmega32 through ISP
- **Breadboard + Jumper Wires** — Circuit assembly, component connection, and hardware testing

Note

All development will be done in C using AVR Libc, without the Arduino framework, allowing direct access to ATmega32 hardware registers — including Timer1 input capture for reading three ultrasonic echoes.

3. System Design

System Block Diagram



Component Roles

3x HC-SR04 Ultrasonic Sensors

- Mounted front, left, right; each returns distance-to-wall
- TRIG pulse $\rightarrow$ ECHO pulse width $\propto$ distance
- A cell wall is flagged present below a calibrated fixed threshold distance
- Timer1 input capture multiplexed across the three sensors for precise timing

L298N Motor Driver Module

- Dual H-bridge module (drives 2 motors)
- Direction control via IN1–IN4 logic pins
- Speed control via ATmega32A PWM on ENA/ENB

DC TT Gear Motors + Optical Encoders

- 3V–6V rated motors (150–200 RPM)
- Each wheel paired with an optical encoder for pulse counting
- Differential drive: 90° turns and cell-to-cell moves measured by encoder pulse count, not timing

Status LEDs

- Indicate current robot state: exploring, dead-end/backtrack, goal reached, speed run

Power System

- 3x 3.7V Li-Ion (3S): 11.1V pack; L7805 steps down to clean 5V logic rail

ATmega32 Pin Assignment

Function	Port	Pins	Description
Front Sonar	PORTB	PB0 (TRIG), PB1/ICP1 (ECHO)	HC-SR04 front wall ranging
Left Sonar	PORTA	PA5 (TRIG), PA6 (ECHO)	HC-SR04 left wall ranging
Right Sonar	PORTA	PA7 (TRIG), PC7 (ECHO)	HC-SR04 right wall ranging
Motor Direction L	PORTD, PORTC	PD7, PC0	Left motor direction control (L298N IN1, IN2)
Motor Direction R	PORTC	PC1, PC2	Right motor direction control (L298N IN3, IN4)
PWM Output L	PORTD	PD4 (OC1B)	Timer1 PWM speed control for left motor (ENA)
PWM Output R	PORTD	PD5 (OC1A)	Timer1 PWM speed control for right motor (ENB)
Left Encoder	PORTD	PD2 (INT0)	Left wheel optical encoder pulse input
Right Encoder	PORTD	PD3 (INT1)	Right wheel optical encoder pulse input
Status LEDs	PORTC	PC3-PC5	Digital outputs for robot-state indicator LEDs
ISP Programming	PORTB	PB5-PB7	MOSI, MISO, SCK interface for AVR ISP
Reset Circuit	—	Pin 9 (RESET)	Active-low reset with 10k Ω pull-up resistor
Clock Source	—	Pins 12, 13	XTAL1/XTAL2 with 16 MHz crystal & 22pF caps

Note: only PB1 uses hardware input capture (ICP1) for the front sonar; left/right echoes are timed via polling. Left/right encoders use the two external interrupt pins (INT0, INT1) for reliable pulse counting.

4. Role of the ATmega32

ATmega32 — Central Processing Unit

The ATmega32 is the **sole controller** of this project. It performs all sensing, mapping, decision-making, and actuation tasks — there is no secondary microcontroller.

Core Tasks Performed by ATmega32

1. **Wall Sensing:** Triggers and times all three ultrasonic sensors each cell
2. **Cell Mapping:** Records discovered walls into an in-memory maze grid
3. **Flood-Fill Computation:** Recomputes distance-to-goal values as new walls are discovered
4. **Motion Planning:** Chooses the next cell to move into based on current flood values
5. **Motor Control:** Executes forward moves and in-place turns using encoder pulse counts for closed-loop accuracy
6. **Run Management:** Tracks exploration vs. speed-run state and dead-end backtracking

ATmega32 Peripheral Usage

Digital I/O (Sonar Trigger/Echo)

- **PORTA/PORTB/PORTC**: TRIG outputs and ECHO inputs for 3 sensors
- Sensors triggered sequentially each cell to avoid cross-talk

Timer1 — PWM & Input Capture

- **OC1B** (PD4) → Left motor PWM speed (ENA)
- **OC1A** (PD5) → Right motor PWM speed (ENB)
- **ICP1** (PB1) → **Input capture for front-sonar echo timing**

GPIO (Motor Control)

- PD7, PC0 → Left motor H-Bridge (IN1, IN2)
- PC1, PC2 → Right motor H-Bridge (IN3, IN4)

External Interrupts (Wheel Encoders)

- **INT0** (PD2) → Left wheel encoder pulse count
- **INT1** (PD3) → Right wheel encoder pulse count
- Pulse counts drive closed-loop distance and turn control

SRAM — Maze State

- Wall bitmap: 4 bits/cell (N/E/S/W) for up to $N \times N$ cells
- Flood-fill distance array: 1 byte/cell
- Fits comfortably within the ATmega32's 2KB SRAM

GPIO (Status LEDs)

- **PC3–PC5**: indicate exploring, dead-end, and goal/speed-run states

Other Features

Watchdog Timer: reset on software hang

5. Maze Mapping and Algorithm

Maze Representation and Flood-Fill

Cell and Wall Representation

- The maze is modeled as an $N \times N$ **grid of cells**, each with up to 4 walls (N, E, S, W)
- Walls are discovered incrementally as the robot visits each cell and reads its 3 ultrasonic sensors
- Each cell's wall state is stored as a **4-bit bitmap**; the full maze fits in well under 1KB of SRAM

Why Flood-Fill?

- Each cell holds an integer **distance-to-goal** value, initialized by Manhattan distance and updated by a BFS-style wave from the goal outward
- When a new wall is discovered, only the affected region is recomputed — **cheap enough to rerun after every move** on an 8-bit MCU
- Guarantees the robot always moves toward a cell on a currently-shortest known path, without needing global replanning from scratch

Operational Cycle: **Run 1 (Exploration):** start from the entry cell, sense walls with the 3 sonars, explore the maze cell by cell, back out of dead ends when hit, and continue until the goal is reached — the map and the shortest known path are remembered in SRAM. **Run 2 (Speed Run):** starting again from the entry cell, follow the remembered shortest path directly to the goal, with no further exploration.

6. Implementation Plan

Expected Outcome

Final Deliverable

A fully assembled autonomous maze-solving robot that:

- Explores an **unknown maze of unit cells** using front/left/right wall sensing
- Builds an internal map and computes the **shortest known path via flood-fill** as it explores
- Reaches the goal cell on its exploration run, then executes a **fast, direct speed run** along the shortest discovered path
- Reliably completes a maze of at least **3×3 cells** within a bounded time and move budget

Stage 1: Project Update 1

Milestones:

- ☐ Assemble chassis, motors, and control circuit
- ☐ Integrate and calibrate 3x HC-SR04 sensors
- ☐ Implement encoder-based forward-move and 90°-turn primitives
- ☐ Sonars detect walls and show status via LEDs

Deliverable: A robot that reliably moves cell-to-cell and detects walls on three sides — the base platform for maze solving

Stage 2: Final

Milestones:

- ☐ Implement maze grid storage and wall-bitmap updates
- ☐ Implement flood-fill computation and incremental updates
- ☐ Implement full exploration run to goal with dead-end handling
- ☐ Implement speed run along the shortest discovered path
- ☐ Full-maze testing, calibration, and LED signaling

Deliverable: Complete maze-solver robot demonstrating exploration, mapping, and a faster speed run

Group Member Responsibilities

Member	Primary Responsibilities
Hasan Ferdous Mahee	Hardware assembly, power system, and sonar integration
Md. Ariful Islam	Motion primitives (turns/moves) and wall-detection calibration
B. M. Sufian Ashraf	Maze representation, flood-fill algorithm, and run-state management
All Members	Testing, documentation, and presentation

7. Use of Other Controllers

No Additional Controllers Used

Design Decision

This project uses **only the ATmega32** microcontroller. No ESP32, Arduino, or Raspberry Pi is required.

Justification:

- Flood-fill on a small grid is cheap enough in both memory and compute for an 8-bit MCU
- Sensor and motor I/O needs (3 sonar pairs, 2 PWM channels, 4 direction pins) fit well within the ATmega32's 32 available pins
- A single controller keeps the system simpler, cheaper, and easier to debug than a dual-controller design
- Direct register-level programming gives full control over sonar timing precision, which matters for reliable wall detection

8. Testing and Success Criteria

Test Environment Setup

- **Maze:** Physical grid maze with uniform unit cells and vertical walls for reliable sonar reflection
- **Maze Size:** 3×3 cells, including one dead end and at least one intersection

Test Procedures:

1. **Unit Testing:** Verify each of the 3 sonar sensors independently against known distances
2. **Module Testing:** Test encoder-based forward-move and turn accuracy over repeated cycles
3. **Integration Testing:** Sonars detect walls correctly and status LEDs reflect robot state
4. **Full-Maze Testing:** Complete exploration run, followed by a speed run; measure both times
5. **Repeatability:** Run 5 consecutive full explore-then-speed-run cycles; record success/failure

Essential Features (Must Have)

- ✓ Correctly detects walls on 3 sides at each cell
- ✓ Correctly detects and backtracks out of dead ends during exploration
- ✓ Completes full exploration of a 3×3 maze and reaches the goal
- ✓ Flood-fill map stays consistent as walls are discovered
- ✓ Executes a speed run strictly faster than the exploration run, following the remembered path directly

9. Challenges and Future Scope

Expected Challenges

- **Sonar cross-talk** between the three simultaneously-mounted ultrasonic sensors
- **Encoder calibration** for consistent pulse-count-to-distance and pulse-count-to-turn-angle mapping
- **Wall-detection thresholding** under sensor noise near cell boundaries
- **SRAM budgeting** for the wall bitmap and flood-fill arrays as maze size grows
- **Dead-end handling** in the exploration algorithm — reliable detection and backtracking

Future Scope

Short-Term Improvements

- Integrate the **optical wheel encoders** (already acquired) for closed-loop odometry
- Add a **gyroscope** for precise turn correction

Long-Term Extensions

- Full **micromouse-style** speed optimization (smooth curves, ramped acceleration)
- **Multi-robot maze solving** with shared map data
- Port to **dynamic/reconfigurable mazes**
- **Machine-learned** move policies as an alternative to flood-fill

Real-World Applications

Search-and-rescue robots, autonomous warehouse navigation, indoor drones, and educational micromouse competitions.

Thank You!

Maze Solver Robot — ATmega32 Project Proposal

Md. Ariful Islam Hasan Ferdous Mahee B. M. Sufian Ashraf